ALUMNO:
José Ricardo Cruz Aguilar

DOCENTE:
José Miguel Carrera Pacheco

ACTIVIDAD:
Comunicación asíncrona accesible

Async JavaScript
El modelo de JavaScript es de un solo hilo (single-threaded), lo que significa que solo puede ejecutar una cosa a la vez. La asincronía permite que tareas de larga duración (como peticiones a bases de datos) no bloqueen el hilo principal, manteniendo la interfaz fluida.

Evolución de la Asincronía:

Callbacks: Funciones que se pasan como argumentos para ejecutarse tras una tarea. Su abuso genera el "Callback Hell".

Promesas (Promises): Objetos que representan el éxito (`resolve`) o fracaso (`reject`) de una operación futura.

Async/Await: Introducido en ES2017, es "azúcar sintáctico" sobre las promesas. Permite escribir código asíncrono que parece síncrono, facilitando la lectura y el mantenimiento.

APIs REST (Representational State Transfer)
REST es un estilo de arquitectura para sistemas distribuidos basado en el protocolo HTTP. Permite la comunicación entre un cliente (frontend) y un servidor (backend) de forma estandarizada.

Principios Fundamentales:

Recursos: Todo se trata como un recurso (usuarios, productos, posts) identificado por una URL única (Endpoint).

Métodos HTTP:

`GET`: Recuperar datos.

`POST`: Crear un nuevo recurso.

`PUT` / `PATCH`: Actualizar recursos.

`DELETE`: Eliminar un recurso.

Estados de Carga Accesibles (Accessible Loading States)

Cuando realizamos peticiones asíncronas, es vital informar al usuario que algo está sucediendo. Sin embargo, para los usuarios de lectores de pantalla (tecnologías de asistencia), un spinner visual es invisible.

Mejores Prácticas de Accesibilidad (A11Y):

ARIA Live Regions: Utilizar el atributo `aria-live="polite"` o `aria-live="assertive"` en el contenedor del mensaje de carga. Esto anuncia automáticamente al lector de pantalla cuando el contenido cambia.

Atributos de Estado: Usar `aria-busy="true"` mientras se cargan los datos y cambiarlo a `false` al finalizar.

Skeleton Screens: Son preferibles a los spinners circulares ya que dan una idea de la estructura de la página, pero deben estar etiquetados correctamente (ej. `aria-label="Cargando contenido..."`).

Manejo de Errores de Red

En el desarrollo web, el error no es una excepción, es una posibilidad constante (caída de internet, servidor lento, error 404). Un buen sistema debe ser resiliente.

Estrategias de Manejo:

Bloques Try/Catch: Esencial al usar `async/await` para capturar errores de ejecución y de red.

Validación de Respuesta: La API de `fetch` no lanza un error en códigos 404 o 500; es necesario verificar la propiedad `response.ok`.

Feedback al Usuario: No basta con un `console.log`. Se debe mostrar un mensaje claro en la UI y, si es posible, un botón de "Reintentar".

Timeouts: Implementar un límite de tiempo para que la petición no se quede "colgada" indefinidamente si la conexión es muy lenta.